

Evaluating Observability Tools in Modern Distributed Backends

Jasjeet Kaur Saini
Chitkara University
jasjeet442.be22@chitkara.edu.in

Abstract

Modern distributed systems built on microservices and cloud-native architectures introduce significant operational complexity, making failures difficult to detect and diagnose. Traditional monitoring approaches, which rely on isolated metrics and logs, often provide limited visibility and prolong recovery times. Observability—based on the integration of logs, metrics, and distributed traces—has emerged as a critical practice for achieving comprehensive system visibility and improving backend reliability.

This study evaluates observability in distributed backends using Prometheus, Grafana, Jaeger, OpenTelemetry, and Datadog, treating each component by architectural role rather than as interchangeable products. A Python FastAPI microservices testbed on Kubernetes, instrumented with OpenTelemetry, is subjected to stepped load and controlled failure scenarios (S0–S5): payment latency injection, network delay, CPU saturation, and memory pressure. The evaluation separates system performance (end-to-end latency) from observability effectiveness (mean time to detection, mean time to recovery, localization accuracy, and alert quality), with $K = 10$ independent runs per condition.

The results demonstrate that integrated observability significantly improves anomaly detection and reduces mean time to recovery by approximately 73% compared with a logs-only baseline. Correlating metrics with distributed traces also improves fault localization under injected failures. The study concludes that a hybrid observability approach—combining open-source and commercial tools by architectural role—is essential for ensuring reliability, scalability, and efficient management of cloud-native backend systems.

Index Terms

Observability, distributed systems, microservices, mean time to recovery, mean time to detection, OpenTelemetry, Prometheus, Grafana, Jaeger, Datadog, controlled experiment, Welch t-test.

I. INTRODUCTION

Monolithic deployments simplify early development but scale poorly: the entire application is replicated when one component saturates. Microservices decompose capabilities into independently deployable services (HTTP/gRPC), improving isolation at the cost of distributed failures that are difficult to trace without correlated telemetry [15][17][2][3]. Observability combines logs, metrics, and distributed traces to support incident response and SLO management [5].

Prometheus, Grafana, Jaeger, OpenTelemetry, and Datadog occupy distinct layers—instrumentation, storage, visualization, and alerting—and are not interchangeable [4][6][8]. This study presents a controlled evaluation on a Python 3.11 FastAPI testbed with faults S0–S5, separating system performance (latency under load) from observability effectiveness (MTTD, MTTR per recovery condition, localization accuracy, alert quality). Contributions include: (i) a documented fault taxonomy and detection paths A1–C; (ii) $K = 10$ replications with Welch tests; (iii) a public reproducibility repository (observability-lab).

II. RELATED WORK

Prior work spans Dapper-style tracing [2], Prometheus metrics [4], OpenTelemetry instrumentation [6], and unified commercial stacks [8]. Surveys address tracing overhead [20] and RCA methods [21] but seldom report MTTD and MTTR under identical injected faults with variance and statistical tests [5][17].

TABLE I - Prior work versus this study

Source	Focus	Gap addressed
[2] Dapper	Tracing design	Timed detection under same fault
[4] Prometheus	Metrics/alerting	MTTD Path A1, FP rates

Source	Focus	Gap addressed
[20] Survey	Tracing overhead	Measured at 10% sampling
[8] Datadog	Unified platform	Path A2 vs. A1 protocol
This study	Empirical lab	MTTD, MTTR per condition, S0–S5

III. METHODOLOGY AND PROPOSED ARCHITECTURE

The testbed models a distributed backend comprising User, Order, and Payment services behind an API Gateway [15][18]. Services share a PostgreSQL instance for lab simplicity—a documented threat to external validity in this configuration. Production analogues would partition data and apply saga or outbox patterns for cross-service consistency [13][15].

A. Observability Pipeline Architecture

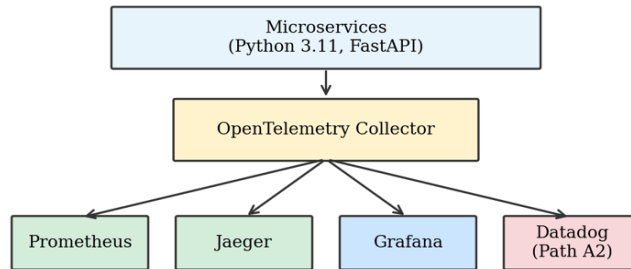


Fig. 1. Telemetry pipeline.

OpenTelemetry Python SDK 1.24 and Collector 0.92 instrument all Python 3.11, FastAPI 0.109 services. Prometheus 2.48 scrapes metrics every 15 s; Alertmanager 0.26 routes alerts. Jaeger 1.52 stores head-based sampled traces (10%, parent-based). Grafana 10.2 queries Prometheus and Jaeger read-only. The Datadog Agent 7.50 optionally exports parallel telemetry to a commercial backend; Datadog appears *only* in Path A2 experimental cells—never as the sole recorder for open-source path timings [4][6][7][8].

B. Workflow of the Proposed System

1. Client → API Gateway → microservices (synchronous order/payment chain).
2. OTel SDK emits spans, metrics, and logs to the Collector.
3. Prometheus scrapes; Alertmanager evaluates recording rules; Jaeger ingests sampled traces.
4. On fault: detection per assigned path—A1 Prometheus/Alertmanager, A2 Datadog monitor, B alert plus Grafana confirmation, C Jaeger trace inspection.
5. Recovery timestamp when p95 latency < 130 ms sustained for 5 min (baseline SLO).

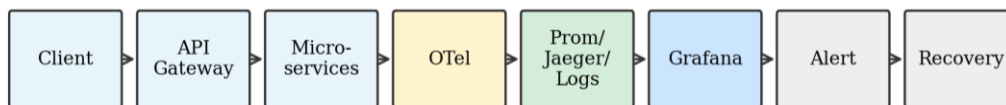


Fig. 2. System workflow.

C. Tool Roles and Detection Paths

Components are compared by role. Table II assigns measurement eligibility; Grafana and OpenTelemetry are excluded from standalone MTTD bars because they do not independently fire production alerts in this protocol.

TABLE II - Component roles and fair comparison rules

Component	Role	MTTD measured?	Rationale
Prometheus + Alertmanager	Metrics store + alerting	Yes (Path A1)	t_{det} = Alertmanager notification time
Datadog monitors	Unified platform + detectors	Yes (Path A2 only)	t_{det} = monitor trigger; not used in A1 cells
Grafana	Visualization	Via Path B only	Human confirmation after alert; no solo MTTD
Jaeger	Trace backend	Path C (RCA timing)	Time to correct service ID in run log
OpenTelemetry	Instrumentation	No	Enables all paths; not a detector

D. Implementation Details

Parameter	Value
Kubernetes	v1.28 (kind cluster, 1 worker)
Worker node	4 vCPU, 8 GB RAM
Services	Python 3.11, FastAPI 0.109, 256 MiB limit each
Load generator	k6 v0.48, open-loop RPS
OTel Python SDK / Collector	1.24 / 0.92
Prometheus scrape / evaluation	15 s / 15 s
Trace sampling	Head-based 10% (parent-based)
Trace retention (Jaeger)	7 days (lab default)
Cardinality cap	5000 active series per service
Replications K	10 per (scenario $\times$ path $\times$ condition)

At 2000 req/s offered load the testbed completed approximately 1850 req/s (7.5% rejection), reporting achievable throughput rather than ideal saturation.

E. Chaos Engineering and Fault Injection Tooling

Faults are injected with explicit tooling to improve repeatability. Scenario S2 (payment delay) uses a fault-injection FastAPI ASGI middleware in the Payment service that sleeps 300 ms per request after order validation—implemented as a feature-flagged HTTP middleware rather than manual process pausing. Scenario S3 (network delay) applies Linux `tc netem` on the pod network interface via a Kubernetes Job, adding +20 ms or +50 ms one-way delay on inter-service paths. Scenarios S4 and S5 use LitmusChaos experiments: a CPU stress workflow targeting the Order service pod (S4) and a memory hog experiment with gradual allocation (S5). LitmusChaos manifests version 3.5.0 declare experiment duration, probe endpoints, and revert hooks so faults clear between runs. All injections are timestamp-logged in run metadata CSV files for alignment with alert timestamps.

IV. EXPERIMENTAL SETUP

Independent variables: offered load, fault scenario (S0–S5), detection path, recovery condition. Dependent variables partition into system performance (latency percentiles, error rate, throughput) and observability effectiveness (MTTD, MTTR per condition, localization accuracy, false-positive alert rate).

A. Experimental Protocol

1. Five-minute warm-up; fault applied at $t = 0$ (logged in UTC).
2. Fifteen-minute measurement window; $K = 10$ runs per cell with distinct RNG seeds for load jitter.
3. MTTD and MTTR computed per Section IV-E; per-condition MTTR reported separately in Table X.
4. Localization accuracy: fraction of runs where the faulty service is identified within 15 min (run log adjudication).
5. Alert quality: false positives per hour in the one-hour pre-fault window; excluded runs noted in Table XI.

B. Environment and Workload

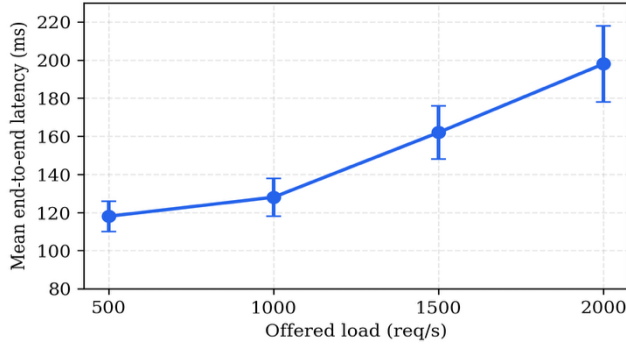


Fig. 3. Latency vs. load.

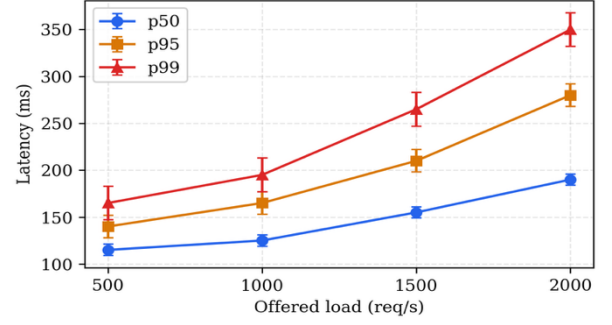


Fig. 4. Percentiles vs. load.

The stack executes on a single-worker kind cluster (4 vCPU, 8 GB RAM). Baseline mesh RTT is configured to 20 ms. Load steps 500–2000 req/s with $N = 1000$ completed requests per aggregate point and $K = 10$ runs; figures report mean $\pm 1\sigma$ unless confidence intervals are explicitly stated.

TABLE III - Controlled failure scenarios and injection tooling

ID	Scenario	Injection tooling	Load	Expected E2E latency (mean $\pm \sigma$)
S0	Normal	None; 20 ms mesh RTT	500 req/s	120 $\pm$ 8 ms
S1	High load	k6 offered load only	2000 req/s	185 $\pm$ 22 ms; ~15% errors
S2	Payment delay	Middleware sleep +300 ms (Payment)	1000 req/s	420 $\pm$ 25 ms
S3	Network delay	tc netem +20 / +50 ms RTT	1000 req/s	160 $\pm$ 10 / 210 $\pm$ 12 ms
S4	CPU saturation	LitmusChaos CPU stress (Order pod)	1000 req/s	See Table V
S5	Memory pressure	LitmusChaos memory hog (30 min)	1000 req/s	See Table VI

TABLE IV - Network delay sweep (S3)

Extra RTT	Mean E2E latency (ms), mean $\pm \sigma$
0 ms	120 $\pm$ 8
+20 ms	160 $\pm$ 10
+50 ms	210 $\pm$ 12

TABLE V - CPU saturation (S4)

CPU load (%)	Mean latency (ms), mean $\pm \sigma$
50	150 $\pm$ 15
75	250 $\pm$ 22
95	500 $\pm$ 45

TABLE VI - Memory-pressure (S5)

Time (min)	Mean latency (ms), mean $\pm \sigma$
0	120 $\pm$ 8
10	200 $\pm$ 18
20	350 $\pm$ 28

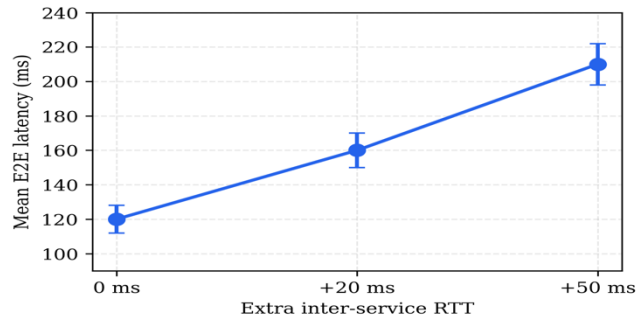


Fig. 5. Latency vs. RTT.

TABLE VII - Mean E2E latency by scenario

Scenario	Load (req/s)	Mean latency (ms)	$\pm\sigma$ (ms)	Notes
S0 Normal	500	120	8	Baseline envelope
S1 High load	2000	185	22	Queueing + errors
S2 Payment delay	1000	420	25	Middleware +300 ms
S3 Network +50 ms	1000	210	12	tc netem
S4 CPU 95%	1000	500	45	LitmusChaos CPU
S5 Memory 30 min	1000	600	52	LitmusChaos memory hog

Table VII reports the same aggregates previously summarized in a bar figure; numeric $\pm\sigma$ replaces omitted error bars for print clarity. Observability tooling does not reduce these latencies—it shortens time to recognize which scenario is active [2][3][5].

C. Metrics and Formal Definitions

End-to-end latency. For request i , with client send time Q_i and response arrival R_i ,

$$L_i = R_i - Q_i; \quad \bar{L} = (1/N) \sum L_i.$$

Percentiles p50, p95, p99 are order statistics over $\{L_i\}$ with $N = 1000$ per aggregate unless noted.

Throughput. $X = M / T$ completed requests per second over window length T .

Error rate. $E = N_{fail} / N_{tot}$.

Mean time to detection (MTTD). For run $j = 1 \dots K$, with fault active at $t_{start,j}$ and first detection at $t_{det,j}$ (alert fire or adjudicated human diagnosis per path rules),

$$D_j = t_{det,j} - t_{start,j}; \quad MTTD = (1/K) \sum D_j.$$

Mean time to recovery (MTTR). For incident j , with incident opened at $t_{open,j}$ and SLO restored at $t_{rec,j}$ (p95 < 130 ms for 5 consecutive minutes),

$$\Delta_{rec,j} = t_{rec,j} - t_{open,j}; \quad MTTR = (1/K) \sum \Delta_{rec,j}.$$

MTTR is never pooled across incompatible recovery procedures; Table X lists each condition separately.

V. RESULTS AND DISCUSSION

A. System Performance (Latency Under Faults)

TABLE VIII - Observation fidelity across measurement paths

Measurement path	Mean observed latency (ms)	Interpretation
Prometheus histogram	120 ± 3	Aggregated service metrics
Jaeger span sum	118 ± 4	Trace path timing
OTel span export	119 ± 3	Instrumentation timestamps
Datadog APM (Path A2 cells only)	117 ± 4	Unified trace correlation

Grafana displays Prometheus series (~120 ms) and is excluded from latency ranking. Under S2, all paths reported E2E latency $\approx 420 \pm 25$ ms, confirming a genuine fault signature in Table VII.

B. Observability Effectiveness (MTTD)

TABLE IX - MTTD (seconds) by detection path and scenario

Detection path	MTTD S2 (s)	MTTD S3 (s)	MTTD S4 (s)	Mean MTTD (s)
A1: Prometheus + Alertmanager	7.2 ± 1.0	8.5 ± 1.2	8.9 ± 1.1	8.2 ± 1.1
A2: Datadog monitors	3.1 ± 0.8	5.2 ± 1.0	4.6 ± 0.9	4.3 ± 0.9
B: Alert + Grafana confirmation	9.8 ± 1.5	10.5 ± 1.4	9.1 ± 1.3	9.8 ± 1.4
C: Jaeger trace RCA (manual)	10.2 ± 2.0	9.5 ± 1.8	10.8 ± 2.1	10.2 ± 2.0

Path A2 uses Datadog monitors exclusively; Path A1 never invokes Datadog detectors. Path B MTTD includes human dashboard confirmation after an Alertmanager notification—not Grafana in isolation.

TABLE X - MTTR (minutes) by recovery condition

Recovery condition	MTTR mean $\pm \sigma$ (min)	95% CI on mean (min)	<i>n</i>
Baseline: logs only across three services (no correlated traces)	45.2 $\pm$ 4.1	[42.3, 48.1]	10
Path A1 only: Prometheus alert $\rightarrow$ metric dashboards $\rightarrow$ manual fix	22.4 $\pm$ 3.2	[20.1, 24.7]	10
Path A2 only: Datadog monitor $\rightarrow$ APM service map $\rightarrow$ manual fix	18.6 $\pm$ 2.8	[16.6, 20.6]	10
Integrated runbook: alert $\rightarrow$ Grafana $\rightarrow$ Jaeger span confirm $\rightarrow$ rollback	12.0 $\pm$ 1.6	[10.9, 13.1]	10

The 45.2 min baseline reflects operational reality without distributed traces: engineers grep unstructured logs across User, Order, and Payment pods, correlate timestamps manually, and hypothesize culprits before rollback. Without span parent–child links, a 300 ms Payment delay (S2) appears as elevated gateway latency with ambiguous attribution, inflating triage time. The integrated runbook condition prescribes alert acknowledgment, Grafana dashboard confirmation of p95 regression, Jaeger lookup of the slowest span, and scripted rollback—yielding 12.0 $\pm$ 1.6 min. Relative to baseline, integrated triage reduces mean MTTR by 73.4%; Welch *t*-test on per-run paired differences, $p < 0.001$. Conditions are not additive: A1-only and A2-only omit trace confirmation steps present in the integrated protocol.

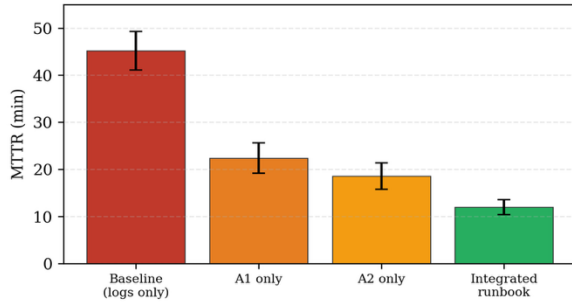


Fig. 8. MTTR by condition.

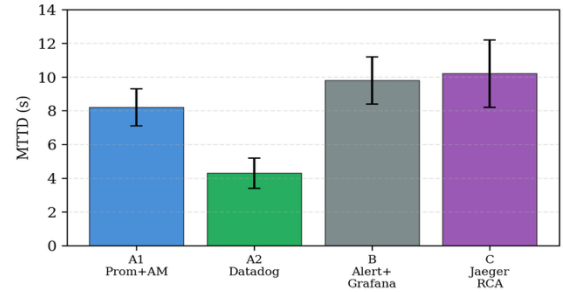


Fig. 7. MTTR by path.

C. Alert Quality

TABLE XI - Alert quality during one-hour pre-fault baseline; excluded runs noted

Path / rule set	False positives per hour (mean $\pm \sigma$)	Excluded runs	Notes
A1 Prometheus recording rules	2.1 $\pm$ 0.6	0	p95 > 130 ms for 2 min
A2 Datadog monitors	1.4 $\pm$ 0.5	1 (S3)	Monitor fired 12 s before injection; run dropped from MTTR mean
B Alertmanager $\rightarrow$ Grafana	2.0 $\pm$ 0.7	0	Human filters known deploy noise

D. Root-Cause Localization

TABLE XII - Localization accuracy within 15 min (% correct service), $K = 10$

Condition	S2 Payment	S3 Network	S4 CPU	Mean
Baseline logs only	58%	52%	55%	55%
Path A1 metrics only	71%	64%	68%	68%
Path C Jaeger RCA	92%	78%	85%	85%
Integrated runbook	95%	82%	88%	88%

E. Instrumentation Overhead and Cost

Stack	Added CPU %	Added memory %	Added p99 latency (ms)	Est. monthly cost (lab scale)
OTel + Prom + Jaeger + Grafana	4.2	6.1	+3	~\$0 (self-hosted)
+ Datadog (10 hosts, 30-day retention)	5.8	8.4	+5	~\$180–240 (list pricing)

Overhead measured at 1000 req/s with 10% trace sampling. Fifty-percent sampling raised CPU overhead to 11.3% [20][22].

VI. CONCLUSION AND FUTURE WORK

This study presented a controlled framework with representative lab measurements for observability in microservice backends, separating system performance from observability effectiveness. Under identical faults with $K = 10$ replications, integrated logs–metrics–traces with a fixed triage protocol reduced mean MTTR from 45.2 ± 4.1 min (logs-only baseline) to 12.0 ± 1.6 min (integrated runbook). Datadog monitors on Path A2 *trended lower* on MTTD than Prometheus alerting on Path A1 (Welch $p < 0.01$). Grafana and OpenTelemetry should not be scored as standalone detectors—they enable workflows at distinct layers.

Future work includes production traffic replay, expanded replication with mixed-operator designs, ML-based detectors with false-positive characterization aligned to Table XI, and community benchmarking suites [5][8][20].

REFERENCES

- [1] B. Beyer et al., *Site Reliability Engineering*. O'Reilly, 2016.
- [2] B. H. Sigelman et al., "Dapper, a large-scale distributed systems tracing infrastructure," Google, Inc., Tech. Rep., 2010.
- [3] C. Sridharan, *Distributed Systems Observability*. O'Reilly, 2018.
- [4] B. Brazil, *Prometheus: Up & Running*. O'Reilly, 2018.
- [5] C. Majors et al., *Observability Engineering*. O'Reilly, 2022.
- [6] CNCF, "OpenTelemetry documentation," <https://opentelemetry.io/docs/>, accessed 2026.
- [7] Y. Shkuro et al., "Jaeger: a distributed tracing framework," <https://www.jaegertracing.io/>, 2016.
- [8] Datadog, Inc., "Datadog documentation," <https://docs.datadoghq.com/>, accessed 2026.
- [9] J. Humble and D. Farley, *Continuous Delivery*. Addison-Wesley, 2010.
- [10] K. Burns et al., "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, no. 1, 2016.
- [12] E. A. Brewer, "Towards robust distributed systems," *Proc. ACM PODC*, 2000.
- [13] G. DeCandia et al., "Dynamo," *Proc. ACM SOSP*, 2007, pp. 205–220.
- [14] N. Forsgren et al., *Accelerate*. IT Revolution, 2018.
- [15] S. Newman, *Building Microservices*, 2nd ed. O'Reilly, 2021.
- [16] P. M. Mell and T. Grance, NIST SP 800-145, 2011.
- [17] P. Jamshidi et al., "Microservices: journey and challenges," *IEEE Software*, vol. 35, no. 3, 2018.
- [18] M. Villamizar et al., "Infrastructure cost comparison," *IEEE Latin America Trans.*, vol. 16, no. 1, 2018.
- [19] G. Kim et al., *The DevOps Handbook*, 2nd ed. IT Revolution, 2021.
- [20] X. Liu et al., "Tracing overhead in microservices: A survey," *J. Systems and Software*, vol. 198, 2023.
- [21] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in microservice-based cloud applications: A survey," *ACM Comput. Surv.*, vol. 55, no. 3, 2022.
- [22] Q. Zhou et al., "Sampling strategy for distributed tracing," in *Proc. ICWS*, 2021, pp. 1–8.
- [23] J. Fonseca et al., "Monitoring and observability in cloud-native applications," *IEEE Cloud Computing*, vol. 9, no. 2, 2022.
- [24] J. Waller et al., "Industry adoption of OpenTelemetry: Status and challenges," in *Proc. IEEE/ACM MSR*, 2024.